Design Decision Record

The Governed Data Product for AI -- Berka (PKDD'99) bank data

Context and problem. Turn eight raw banking files (1,079,680 rows: 1.06 m transactions, 5,369 clients, 4,500 accounts, 682 loans) into a trusted, documented, entitlement-aware data product that an AI assistant can query correctly, and answer the ten business questions through it. The brief warns against infrastructure work, so the stack is one PostgreSQL database, numbered SQL files, plain Python and a Makefile (`scripts\windows.ps1` on native Windows). Labels: [RUN] executed, [CODE] in the source, [LLM] model output, [REC] recommended but not built, [ASSUMED] awaiting business confirmation.

Decisions

1. PostgreSQL, with the pipeline as SQL files plus Python orchestration. *Options:* a cloud warehouse with dbt and an orchestrator; DuckDB in-process; PostgreSQL. *Choice:* PostgreSQL (16 on the Linux/Docker build, 18 on the native Windows build; identical DDL). *Why:* Task 3 is enforcement, and the only honest way to show that a generated query cannot bypass an entitlement filter is to put the filter in the engine and prove it by connecting as a restricted user. DuckDB has no row level security, so that task would have become prose. *Cost:* no built-in lineage or scheduling, and a single node; acceptable at this volume and off the critical path of the brief.

2. Bronze, silver, gold -- and cleaning that never repairs. `raw` holds the files verbatim as text, hashed per file; `stg` types, decodes and resolves the transaction sign once, and filters nothing; the rule layer grades defects; `core` is the model; `ai` is the curated surface. *Options:* repair bad values; drop defective rows; or transform without overwriting and let a graded rule layer decide what reaches the product. *Choice:* the third. *Why:* a repaired value is indistinguishable from an observed one a week later, and a silent drop changes every denominator. Of 21 cleaning steps one removes a row (quarantine, 1 row), and core = staging - quarantine is checked on rows and money after every build [RUN]. Every changed value survives in a `_raw` column. *Cost:* the product carries flagged rows whose caveat a consumer must read.

3. Star schema with a bridge, plus curated views. *Options:* one wide table at posting grain; Data Vault; the source model. *Choice:* four dimensions, one bridge, two base facts and a derived month-end balance fact. *Why:* the ten questions have four grains (client, account, loan, posting); a wide table turns every count into `count(distinct ...)`, the construct generated SQL gets wrong quietly. Data Vault suits many sources with history; there is one source and one extract. *Cost:* joins, each declared in the semantic layer with a measured cardinality, and a derived fact to rebuild. *Known limitation, not changed in this delivery:* the month-end fact breaks same-day ties by the highest `trans_id`, which is not chronological (A14, caveat C2). The Q9 average is robust to it (about 2 CZK under a chain-based rule), individual account balances are not [REC].

4. Three severities, each with a mechanical consequence. `block` quarantines the row, `warn` keeps it with a flag that travels into the model and the semantic layer, `info` standardises and logs. 124 rules cover completeness, uniqueness, referential integrity, validity, cross-table consistency and reconciliation; 19 fire as 14 defect types, 1 row is quarantined and 440 are flagged [RUN]. *Why:* a severity without a consequence is a comment. The 14 zero-amount postings are `info`: at `block` the naive rule would quarantine valid interest postings and break 14 balance chains to fix nothing. One duplicate transaction by business key is `block`: it double counts a flow and no primary key catches it. *Cost:* three code paths and a quarantine table; in exchange every exclusion is reversible, countable and attributable to a rule.

5. The semantic layer is a validated artefact, not prompt text. *Options:* the schema in the prompt; a BI tool's metrics layer; a file the pipeline checks. *Choice:* one YAML file -- entities with their grain, certified metrics with their time column, joins with cardinality, a glossary of what each term is *not*, and the data quality caveats with counts -- validated against the live database on every build. A non-existent or undescribed column, a false grain, a wrong join cardinality, a stale row count or a metric built on a column that does not exist fails the build [RUN]. Definitions live here once: a problematic loan is status B or D, 76 of 682 = 11.14% (A4 [ASSUMED]). *Why:* a drifted layer gives the model confident wrong information it cannot check. *Cost:* it is maintained alongside the model; the validator makes that safe.

6. Entitlements are row level security on the base tables, keyed on `session_user`. *Options:* instruct the model to add the filter; filter in an application gateway; policies in the database. *Choice:* policies, with the SQL guard as a separate second line. *Why:* asking the model to filter itself is a request, not a control, and a gateway is defeated by one unanticipated query shape. A policy sits inside the plan of every statement; the model only emits text and the predicate is not in the text. Identity is the database role a two-factor login authenticated, not a session variable. Access is derived from data: ownership and portfolios grant clients, a disposition grants that one account and never its owner (D023, D024), delegations expire. Unrestricted access is a portfolio of every client, not an exemption (`rolbypassrls = false`). Nine bypass attempts and the cross-client attempts are refused in the test suite [RUN]. Onboarding is inserts and one credential; offboarding is one `UPDATE`. *Cost:* a database role per user, which conflicts with naive connection pooling (a pool per role, or `SET ROLE` by a trusted gateway). A real bank also needs a read audit log and field masking [REC].

7. The LLM is an untrusted query author. It has no tools, no filesystem and no database: it sees the rendered semantic layer and the question. Its SQL is parsed with sqlglot and must be one read-only `SELECT` on `ai.*`, calling no function outside the allowed schemas, under a row cap it cannot raise, a statement timeout and a read-only transaction, on the persona's own two-factor connection [CODE][RUN]. A name the schema does not have fails in the database and is reported, never answered. *Limit:* passing the guard proves a query is safe, not that it answers the question; the intent verifier that would check meaning is a prototype only [REC]. *Evidence:* Claude Sonnet 5.5 answered all ten and 10 of 10 match the hand-written reference [LLM, recorded 2026-10-25, before the semantic audit changed the pack]. Ollama with qwen2.5-coder:3b is an optional free local provider; its run, kept separate, scored 6 match, 2 partial, 2 mismatch [LLM]. *Cost:* every table must be schema-qualified.

What breaks at 100,000x the volume

Reasoned, not benchmarked: 100 billion postings, 450 million accounts, about 10 TB of facts before indexes.

1. **Ingestion and rebuilds.** `core` is rebuilt with `CREATE TABLE AS` on every

run: hours of rewriting unchanged data. It becomes incremental: facts range-partitioned by month, only open partitions rebuilt, merge not replace.

2. **The data quality runner.** 124 full scans, violating keys sampled through

Python. Rules become partition-scoped (check what arrived), keys are written by `INSERT ... SELECT`, the balance reconciliation keeps per-account state, and `info` rules are sampled with a stated confidence; `block` stays exact.

3. **The month-end fact.** 450 m accounts x 72 month ends is 32 bn rows rebuilt

from scratch. It becomes a snapshot maintained from the previous month plus that month's postings.

4. **Latency and concurrency.** Generated ad-hoc SQL over 10 TB is a full scan

unless it prunes; many users exhaust one node and a role-per-user pool. Partitions and indexes on the certified time columns, pre-aggregated marts for certified metrics, per-tier timeouts and a pooled gateway doing `SET ROLE`.

5. **The entitlement predicate.** A hash join over 4,500 accounts today; over

millions for a large portfolio. It becomes a materialised `(role, account_id)` table, refreshed on change and co-partitioned with the facts. The design does not change: still a policy, onboarding still an insert.

6. **Evaluation and operations.** Reference queries that scan everything are

too slow to run on every build, and ten questions are not a test set: frozen samples with checksums, a hold-out question set, and monitoring of guard refusals, query cost, rule trends and entitlement changes.

What does not break is the place the filter sits and the semantic layer's size: about 8,400 tokens at any volume, though many more entities would need retrieval instead of one pack. A single PostgreSQL node is the wrong host for 10 TB; PostgreSQL with Citus, or a columnar MPP engine such as ClickHouse or StarRocks, carrying the same schemas, rules and policy model, is the port to evaluate.

Assumptions and residual risks. [ASSUMED]: problematic = B or D (A4), interest postings count as incoming money (A7), every amount is CZK (A19), calendar not fiscal. Residual: the recorded Claude run predates the current semantic pack; the intent verifier and the month-end tie-break fix are not built; there is no read audit; and the second-factor lockout counts only failures the client commits, so a direct connection holding a stolen password can keep guessing.

Generated by `make ddr` at 2026-10-28T16:11:54+00:00 (git b5b18f2). Source text: deliverables/DDR_source.md